

Former Robot 메뉴얼

ROS2 Humble, Jetson AGX Orin Docker 이용 버전

Ver 1.0

2023. 4. 27.

(주)로아스

목차

1. 일반사항	3
1.1. 전원켜기	3
1.2. 전원끄기	3
1.3. 조이스틱 조작	3
1.4. 로봇 수동 충전하기	4
2. 로봇 사양 및 시스템 구성	6
2.1. 로봇사양	6
2.2. Docker 구성	7
2.3. 사용자 PC 연결	7
3. ROS_DOMAIN_ID 관련	8
3.1. ROS_DOMAIN_ID 변경	8
3.2. DDS 연결 인터페이스 변경	9
3.2.1. 유선랜포트 사용 (eth0)	9
3.2.2. 무선랜포트 사용 (wlan0)	10
4. rviz2 사용	12
4.1. 모니터 연결	12
4.2. DISPLAY 환경변수 확인	12
4.3. rviz2 실행	12
5. Former 로봇 ROS 관련 정보	13
5.1. Topic 정보	13
5.2. Service 정보	16
5.3. Action 정보	16

1. 일반사항

1.1. 전원켜기

로봇 후면의 Emergency 버튼을 누른다. 로봇 후면의 보호 커버를 열고, Power Switch 버튼을 눌러 전원을 켜다.



로봇의 부팅 과정이 완료되면, 아래의 그림과 같이 로봇의 램프가 빨간색으로 깜빡인다. 후면의 Emergency 버튼을 시계방향으로 회전하여 릴리즈하고, 로봇의 램프가 하늘색으로 되는 것을 확인한다.

1.2. 전원끄기

로봇 후면의 보호 커버를 열고, Power Switch를 눌러 전원을 끈다.

1.3. 조이스틱 조작

로봇과 같이 납품된 조이스틱을 이용하여 로봇을 수동 조정할 수 있다. 연결된 조이스틱 확인을 위해 로봇 후면 커버 안의 시리얼번호를 확인한다.





로봇과 같이 동봉된 조이스틱 후면의 시리얼번호를 확인하여 로봇과 동일한 번호인지 확인한다. 조이스틱의 전면 가운데 버튼을 눌러 전원을 켜다. 조이스틱의 램프가 파란색으로 고정된 상태라면 로봇과 연결이 완료된 것이다.



로봇의 Emergency 버튼이 릴리즈되어 있는지 확인한다. 조이스틱의 왼쪽 L1 버튼을 누른 상태로, 좌측 레버 (전진/후진), 우측 레버 (반시계방향/시계방향 회전)을 이용하여 로봇을 움직인다.

1.4. 로봇 수동 충전하기

로봇의 Emergency 버튼을 누른다. 후면의 보호 커버를 열고, 로봇과 같이 공급된 수동 충전기를 꼽는다.



2. 로봇 사양 및 시스템 구성

2.1. 로봇사양



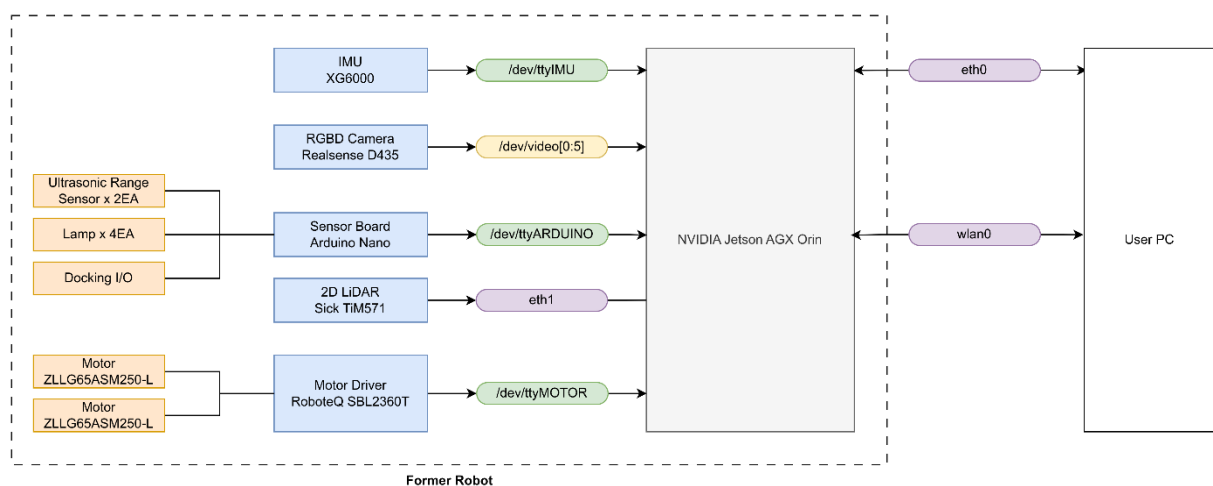
- Mechanical

Size (L X W X H)	404 x 404 x 308 (mm)
Weight	35 (kg)
Ground clearance	60 (mm)

- Performance

Max. speed	1.5 (m/s)
Max. Payload	70 (kg)
Type of drive	Two wheels Differential Drive
Drive power	300W
Working environment	Indoor

- 디바이스 연결 다이어그램



2.2. Docker 구성

현재 Jetson Orin이 Ubuntu 22.04를 지원하지 않는 관계로, ROS2 Humble 버전을 사용하기 위해선 Docker (with nvidia-container)를 사용해야 한다.

Former의 경우 realsense 카메라를 위한 컨테이너 (realsense_humble:1.0), 로봇 제어를 위한 컨테이너 (former_docker:1.0)을 사용한다.

현재 설치되어 있는 이미지를 보려면 다음과 같이 입력한다.

```
$ docker images
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
former_docker        1.0         b0d4baaa0c5c   18 hours ago   5.7GB
realsense_humble     1.0         52146eb76a28   19 hours ago   15.3GB
```

또한 로봇 부팅 시 위 두 개의 컨테이너가 자동 실행되도록 설정되어 있다. 현재 실행되어 있는 컨테이너를 보려면 다음과 같이 명령어 입력한다.

```
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         NAMES
2baa8cd9dd34   realsense_humble:1.0               "/ros_entrypoint.sh ..." 2 hours ago   Up
2 hours                                             realsense_docker
75f9363681f6   former_docker:1.0                  "/ros_entrypoint.sh ..." 2 hours ago   Up
2 hours                                             former_robot
```

2.3. 사용자 PC 연결

Former는 사용자 PC 혹은 확장기기 연결을 위해 USB 2포트, Ethernet 1포트를 제공합니다.

Former 상단 우측의 보호커버를 열면, Ethernet 포트와 USB 1포트가 존재합니다.

Ethernet 포트는 출고 시, 192.168.127.200 고정IP로 설정되어 있습니다. USB1포트는 USB3.1 허브를 통해 Orin과 연결되어 있습니다.

전원 단자도 제공되는데, 각각 24V, 12V, 배터리전압이 제공되며, 각 포트당 3A까지 사용이 가능합니다.

3. ROS_DOMAIN_ID 관련

현재 Former의 경우, 출고 시 ROS_DOMAIN_ID를 4로 고정해두었고, 연결 인터페이스를 eth1 (내부 사용 인터페이스)로 설정되어 있다.

3.1. ROS_DOMAIN_ID 변경

ROS_DOMAIN_ID를 변경하기 위해서는 부팅 스크립트를 수정해야 한다. 두 개의 파일을 수정해야 한다. 터미널을 열고 다음과 같이 입력한다.

```
$ sudo vi /etc/systemd/system/former_bringup.service

[Unit]
Requires=dev-ttyARNDUINO.device dev-ttyIMU.device dev-ttyMOTOR.device
After=dev-ttyARNDUINO.device dev-ttyIMU.device dev-ttyMOTOR.device
realsense_bringup.service

[Service]
RemainAfterExit=yes
ExecStart=/usr/bin/nvidia-docker run --rm -i --net=host --name former_robot -e
ROS_DOMAIN_ID=4 --privileged -v /dev:/dev -v /home/former/dev_ws/src:/root/dev_ws/src:z
-v /home/former/cyclonedds_conf:/root/cyclonedds_conf former_docker:1.0 ros2 launch
former_bringup bringup_robot.launch.py
ExecStop=/usr/bin/nvidia-docker stop former_robot

[Install]
WantedBy=multi-user.target
```

7번 라인의 ROS_DOMAIN_ID 변수의 값을 원하는 값으로 수정하고 저장한다.

```
$ sudo vi /etc/systemd/system/realsense_bringup.service

[Unit]
Requires=dev-video0.device
After=dev-video0.device cyclonedds_tunes.service

[Service]
RemainAfterExit=yes
ExecStart=/usr/bin/nvidia-docker run --rm -i --net=host --name realsense_docker -e
NVIDIA_DRIVER_CAPABILITIES=all -e NVIDIA_VISIBLE_DEVICES=all -e ROS_DOMAIN_ID=4 --
privileged -v /dev:/dev -v /home/former/cyclonedds_conf:/root/cyclonedds_conf
realsense_humble:1.0 /bin/bash -c "source /opt/ros/humble/install/setup.bash && source
/root/realsense_ws/install/setup.bash && export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp &&
export CYCLONEDDS_URI=file:///root/cyclonedds_conf/cyclonedds.xml && ros2 launch
realsense2_camera rs_launch.py enable_sync:=true enable_pointcloud:=true
filters:=decimation color_width:=640 color_height:=480 color_fps:=30 depth_width:=848
depth_height:=480 depth_fps:=30"
ExecStop=/usr/bin/nvidia-docker stop realsense_docker

[Install]
WantedBy=multi-user.target
```

7번 라인의 ROS_DOMAIN_ID 변수의 값을 위에서 설정한 값과 동일한 값으로 수정하고 저장한다.

수정이 완료된 이후, systemd 서비스 데몬을 reloading 하고, 로봇을 재부팅한다.

```
$ sudo systemctl daemon-reload
$ sudo reboot
```

3.2. DDS 연결 인터페이스 변경

Former의 경우 내부적으로 CycloneDDS를 사용한다. 출고 시 DDS 인터페이스를 eth1 (내부 사용 인터페이스)로 되어 있다.

사용자가 사용자PC와 연결을 원할 경우, 4.1에서 진행한 ROS_DOMAIN_ID를 수정하고, 반드시 DDS 인터페이스를 설정하여야 한다.

3.2.1. 유선랜포트 사용 (eth0)

로봇의 외부 유선랜 포트와 사용자PC를 연결한다. 터미널을 열고, 다음과 같이 입력하여 설정 파일을 수정한다.

```
$ vi ~/cyclonedds_conf/cyclonedds.xml

<?xml version="1.0" encoding="UTF-8" ?>
<CycloneDDS                                xmlns="https://cdds.io/config"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://cdds.io/config
https://raw.githubusercontent.com/eclipse-
cyclonedds/cyclonedds/master/etc/cyclonedds.xsd">
    <Domain id="any">
        <General>
            <Interfaces>
                <NetworkInterface            autodetermine="false"
name="eth1" priority="default" multicast="default" />
            </Interfaces>
            <AllowMulticast>default</AllowMulticast>
            <MaxMessageSize>65500B</MaxMessageSize>
        </General>
    </Domain>
</CycloneDDS>
```

6번 라인의 NetworkInterface 항목에서 name 부분을 eth0로 수정하고 저장한다. 이후 로봇을 재부팅한다.

3.2.2. 무선랜포트 사용 (wlan0)

로봇을 사용하려는 무선랜AP에 연결한다. 터미널을 열고, 다음과 같이 입력하여 설정 파일을 수정한다.

```
$ vi ~/cyclonedds_conf/cyclonedds.xml

<?xml version="1.0" encoding="UTF-8" ?>
<CycloneDDS xmlns="https://cdds.io/config"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://cdds.io/config
https://raw.githubusercontent.com/eclipse-
cyclonedds/cyclonedds/master/etc/cyclonedds.xsd">
    <Domain id="any">
        <General>
            <Interfaces>
                <NetworkInterface autodetermine="false"
name="wlan0" priority="default" multicast="default" />
            </Interfaces>
            <AllowMulticast>default</AllowMulticast>
            <MaxMessageSize>65500B</MaxMessageSize>
        </General>
    </Domain>
</CycloneDDS>
```

6번 라인의 NetworkInterface 항목에서 name 부분을 wlan0로 수정하고 저장한다. 이후 로봇을 재부팅한다.

주의사항!

DDS 인터페이스를 eth0, wlan0로 변경하는 경우, 로봇 부팅 시 해당 인터페이스가 **반드시 연결되어 있는 상태**여야 한다. 그렇지 않을 경우, DDS 관련한 부분에서 에러가 발생하고, 로봇의 Docker가 제대로 실행되지 않는다.

4. 로봇 속도 관련 설정

로봇의 최대속도, 가감속도 설정을 패키지에서도 사용자가 수정할 수 있도록 수정하였습니다.

- former_robot/former_description/urdf/former_ros2_control.urdf.xacro

11 라인부터

```
<hardware>
  <plugin>former_hardware_interface/FormerSystemHardwareInterface</plugin>
  <param name="port_name">/dev/ttyMOTOR</param>
  <param name="baudrate">115200</param>
  <param name="robot_acceleration">6000</param>
  <param name="robot_deceleration">6000</param>
</hardware>
```

모터드라이버 설정을 위한 가감속도로서 단위는 0.1RPM입니다.

- former_robot/former_bringup/config/controllers_former.yaml

43번 라인부터

```
linear.x.has_velocity_limits: true
linear.x.has_acceleration_limits: true
linear.x.has_jerk_limits: false
linear.x.max_velocity: 1.5
linear.x.min_velocity: -1.5
linear.x.max_acceleration: 2.5
linear.x.min_acceleration: -2.5
# linear.x.max_jerk: 0.0
# linear.x.min_jerk: 0.0

angular.z.has_velocity_limits: true
angular.z.has_acceleration_limits: true
angular.z.has_jerk_limits: false
angular.z.max_velocity: 2.5
angular.z.min_velocity: -2.5
angular.z.max_acceleration: 3.2
angular.z.min_acceleration: -3.2
# angular.z.max_jerk: 0.0
# angular.z.min_jerk: 0.0
```

선속도의 단위는 m/s, 각속도의 단위는 rad/s 입니다.

5. rviz2 사용

로봇이 현재 실행되고 있는 상태를 확인하기 위해, rviz2를 실행할 경우 다음과 같이 입력한다.

5.1. 모니터 연결

Former의 HDMI 단자를 이용하여 모니터를 연결한다. 부팅이 완료된 이후, 터미널을 열어 Docker에서 X윈도우를 접근할 수 있도록 다음과 같이 명령어를 실행한다. (부팅 후 한 번만 하면 됨)

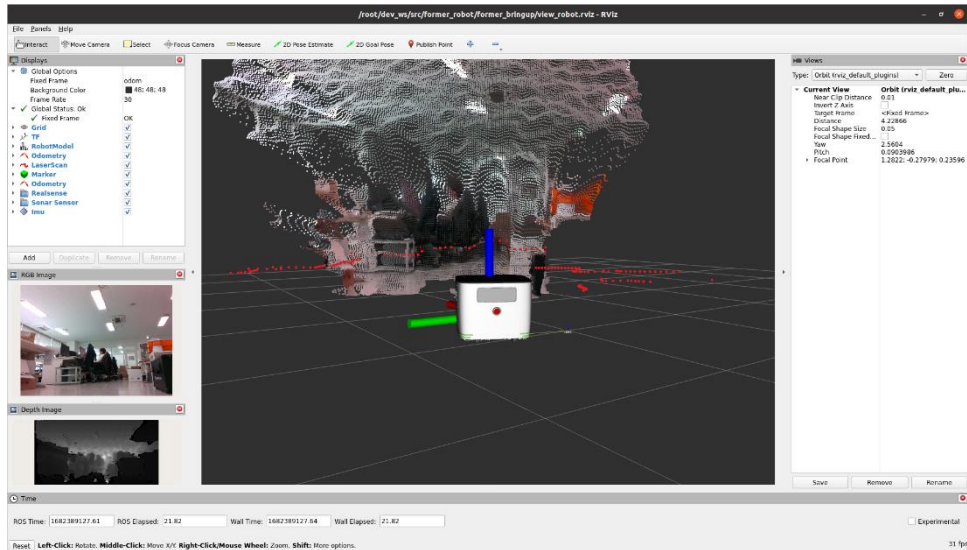
```
$ xhost +local:docker
non-network local connections being added to access control list
```

5.2. DISPLAY 환경변수 확인

```
$ echo $DISPLAY
:0
```

5.3. rviz2 실행

```
$ nvidia-docker run --rm -it --net=host -e NVIDIA_DRIVER_CAPABILITIES=all -e
NVIDIA_VISIBLE_DEVICES=all -e DISPLAY=unix:0 -v /tmp/.X11-unix/:/tmp/.X11-unix -
e ROS_DOMAIN_ID=4 -v /home/former/dev_ws/src:/root/dev_ws/src:z -v
/home/former/cyclonedds_conf:/root/cyclonedds_conf:z former_docker:1.0 rviz2 -d
/root/dev_ws/src/former_robot/former_bringup/view_robot.rviz
```



6. Former 로봇 ROS 관련 정보

6.1. Topic 정보

정상적으로 로봇이 실행되어 있는 경우 전체 토픽 리스트는 다음과 같다.

```
/base_controller/cmd_vel_unstamped
/base_controller/odom
/base_controller/transition_event
/camera/color/camera_info
/camera/color/image_raw
/camera/color/image_raw/compressed
/camera/color/image_raw/compressedDepth
/camera/depth/camera_info
/camera/depth/color/points
/camera/depth/image_rect_raw
/camera/depth/image_rect_raw/compressed
/camera/depth/image_rect_raw/compressedDepth
/camera/extrinsics/depth_to_color
/cloud
/diagnostics
/dock_state
/dynamic_joint_states
/former_io_controller/enable_motor
/former_io_controller/robot_feedback
/former_io_controller/transition_event
/front_lidar/encoder
/front_lidar/imu
/imu_raw
/joint_state_broadcaster/transition_event
/joint_states
/joy
/joy/set_feedback
/l_sonar_range
/line_markers
/parameter_events
/r_sonar_range
/robot_description
/rosout
/scan
/tf
/tf_static
```

- `/base_controller/cmd_vel_unstamped` [*geometry_msgs/msg/Twist*]
Former의 베이스를 움직이기 위한 토폭
- `/base_controller/odom` [*sensor_msgs/msg/Odometry*]
Former의 Odometry 정보
- `/camera/color/image_raw` [*sensor_msgs/msg/Image*]
D435 카메라의 Color 이미지 토폭
- `/camera/depth/color/points` [*sensor_msgs/msg/PointCloud2*]
D435 카메라의 PointCloud2 토폭
- `/camera/depth/image_rect_raw` [*sensor_msgs/msg/Image*]
D435 카메라의 Depth 이미지 토폭
- `/dock_state` [*std_msgs/msg/Bool*]
Former 도킹 상태 (true/false)
- `/former_io_controller/enable_motor` [*std_msgs/msg/Bool*]
Former 로봇 토폭 제어 (true/false)
- `/former_io_controller/robot_feedback` [*former_interfaces/msg/RobotFeedback*]
Former 로봇의 상태 정보
- `/scan` [*sensor_msgs/msg/Scan*]
Front LiDAR의 scan 데이터
- `/imu_raw` [*sensor_msgs/msg/Imu*]
IMU 센서 정보
- `/joint_states` [*sensor_msgs/msg/JointState*]
Former 로봇 Joint 정보
- `/joy` [*sensor_msgs/msg/Joy*]
조이스틱 연결시 조이스틱 입력 정보
- `/l_sonar_range` [*sensor_msgs/msg/Range*]
전면 왼쪽 초음파 센서 정보
- `/r_sonar_range` [*sensor_msgs/msg/Range*]
전면 오른쪽 초음파 센서 정보

6.2. Service 정보

- /set_lamp [former_interfaces/srv/SetLamp]

```
uint8 color
uint8 mode
---
bool success
```

color

```
orange = 0,
magenta = 1,
red = 2,
green = 3,
blue = 4,
mint = 5
```

mode

```
normal = 0,
blink = 1,
breath = 2
```

6.3. Action 정보

- /docking

```
uint8 mode
---
bool success
string message
---
bool is_moving
```

mode

```
docking = 0
undocking = 1
```